National University of Computing and Emerging Sciences
Department of Computing

Student Portal System

Final Combined Project Report

Sprint 1, Sprint 2, and Sprint 3

Submitted By:

Syed Muhammad Mustafa (23I-0661)
Muhammad Talha Qamar (23I-0013)
Muhammad Bin Taimur (23I-0810)

Submitted To:

Mam Rabail Zahid
Section: F
Dated: 29/04/2026

1. Introduction

The Student Portal System is a Java-based business management system developed to digitally manage academic activities within a university. It centralizes student academic operations such as course registration, transcript viewing, attendance management, fee processing, teaching workflows, administrative provisioning, and anonymous feedback. The system follows the Scrum agile methodology and was delivered across three sprints, each adding a clearly scoped slice of functionality on top of a shared domain model and service layer.

Sprint Overview

- Sprint 1 established the student-facing course lifecycle: Course Registration, Course Drop, View Registered Courses, and View Transcript with GPA. It set up the database, the domain model (students, courses, enrolments, grades) and the core service layer.
- Sprint 2 extended the platform with academic self-service analytics and financial transparency: Attendance Tracking, Marks Breakdown, and Fee Challan. Students gained real-time insight into trends, category-wise performance, and itemised dues.
- Sprint 3 completed the system by adding the Faculty and Administrator roles, Role-Based Authentication, and an Anonymous Student Feedback loop, transforming the application from a single-role student utility into a fully integrated three-role academic management platform.

Sprint 3 marks the overall completion of the system. All planned user stories across the three sprints are delivered, the schema supports all three roles end-to-end, and the system is covered by a 65-test white-box suite (100% pass rate) and a 76-case black-box test specification.

2. Project Vision

The vision of the Student Portal System is to provide a centralised, efficient, and user-friendly academic management platform that:

- Automates manual academic and administrative processes.
- Reduces paperwork and administrative workload.
- Provides transparency in course enrolment, grading, attendance, and finance.
- Enables students, faculty, and administrators to manage their academic and operational responsibilities independently.
- Surfaces real-time academic analytics and a closed feedback loop between students and instructors.

Vision Refinement Across Sprints

Sprint 1 framed the vision around centralisation and student self-service. Sprint 2 refined it to emphasise real-time analytics, transparency in grading and fees, and informed academic decision-making. Sprint 3 completed the refinement by moving the platform from a single-role student portal into a three-role integrated system: the application is now role-aware (Student, Faculty, Admin) with each stakeholder seeing a tailored workspace, administrative workflows have moved inside the system, and the feedback loop between students and instructors is anonymised and closed end-to-end.

3. Intended Use and Stakeholders

Stakeholders

- Students

- Faculty / Instructors
- Academic Administration
- University Management

Final Stakeholder Needs

Students

- Register, drop, and view enrolled courses (Sprint 1).
- View transcript and GPA, and track academic progress (Sprint 1).
- Monitor attendance trends, inspect marks breakdown, and view the fee challan (Sprint 2).
- Submit anonymous, one-per-section faculty feedback on a 1 to 5 scale across five standardised questions (Sprint 3).
- Log in through a role-aware login route and access only student-permitted pages.

Faculty / Instructors

- Log in through a faculty-specific login route and land on a faculty dashboard.
- View all assigned course-sections created by the Admin.
- Upload attendance per session using a Present / Late / Absent matrix with colour-coded cells and automatic student roster sorting.
- Upload marks by assessment category (Quiz, Assignment, Project, Sessional, Final Exam) with configurable totals and absolute weights.
- View anonymous student feedback insights per course-section, including overall rating and per-question averages, without student identities.

Academic Administration

- Log in through an admin-specific route and land on an admin dashboard with system-wide stat cards.
- Assign instructors to course-sections for a given term, with protection against duplicate assignments.
- Manage student profiles (search by roll number; edit name, email, major, current semester).
- Manage faculty profiles (select from roster; edit name, email, department, designation).
- Maintain data integrity across enrolment, attendance, marks, and feedback tables.

4. Features and Functionality

4.1 Sprint 1: Course Lifecycle Module

- Course Registration with seat-availability validation.
- Course Drop functionality.
- View Registered Courses.
- View Transcript with automatic GPA calculation.
- Database integration for storing student-course relationships.

4.2 Sprint 2: Academic Analytics and Financial Transparency

Attendance Tracking

- Course-wise attendance percentage.
- Daily attendance ledger.
- Highlights at-risk courses.
- Trend insights.

Marks Breakdown

- Per-course performance.
- Category-wise marks (assignments, quizzes, sessionals, final exam).
- Weighted contribution calculation.
- Performance-risk highlighting.

Fee Challan

- Latest fee challan with itemised breakdown.
- Total amount and registered credit hours.
- Refresh and print options.

4.3 Sprint 3: Roles, Admin, Faculty, and Feedback

Role-Based Authentication and Entry

- Role Selection landing page with three buttons (Student / Faculty / Admin).
- Role-specific Login view: the same login form shows a role-specific subtitle and validates credentials against the correct user table.
- AllowedRoles annotation applied to every view to declare authorised roles.
- RoleRouteResolver and MainLayout.beforeEnter enforce role guards and redirect signed-in users to their role-specific home.
- Session-bound user context via SessionService.

Admin Module

- Admin Dashboard with stat cards (total students, total faculty, total course-section assignments).
- Assign Instructors: pick Course, Section, Term, and Faculty; save creates an assignment row. Duplicate course+section assignments are blocked.
- Manage Users: tabbed view where admins can search students by roll number and edit profile fields, or select a faculty member and edit department / designation.

Faculty Module

- Faculty Dashboard with stat cards for assigned sections and feedback dashboards.
- Upload Attendance: a date by student matrix with P / L / A drop-downs, auto-sorted by roll number, colour-coded, saved transactionally.
- Upload Grades: an assessment-column by student matrix; faculty add columns with category, total marks, and absolute weight; rows are editable and saved as assessment_records.
- Anonymous Feedback Insights: aggregated ratings per course-section with per-question averages and response counts; no student identities exposed.

Student Feedback Module

- Students select an enrolled course-section and submit a 1 to 5 rating on five standardised questions covering instructor clarity, relevance, participation, fairness, and overall satisfaction.
- Submissions are anonymous and one-per-student-per-assignment; re-submission replaces the previous response.

4.4 How Each Sprint Improved the System

Sprint 2 added analytical insight, data transparency, financial visibility, and improved decision-making support. Sprint 3 then closed the end-to-end loop that the prior sprints had left open: before Sprint 3, the attendance, marks, and fee data displayed by Sprint 2 had to be assumed to arrive from somewhere. Sprint 3 fills that gap by introducing the Faculty role that produces attendance and marks data through the upload matrices, and the Admin role that provisions the course-to-faculty assignments that make those uploads possible. Students then consume those live values in the Sprint 2 views and produce

feedback for the faculty, completing the data loop Admin to Faculty to Student to Faculty.

The role-based authentication layer introduced in Sprint 3 is what makes all three roles safely coexist in one application. Route guards via AllowedRoles and MainLayout.beforeEnter prevent cross-role URL tampering, and the RoleRouteResolver delivers each user to the correct dashboard.

5. User Stories

5.1 Sprint 1: Course Lifecycle

- US-1 Register Course: As a student, I want to register for a course so that I can enroll in my semester subjects.
- US-2 Drop Course: As a student, I want to drop a registered course so that I can modify my semester schedule.
- US-3 View Registered Courses: As a student, I want to view my registered courses so that I can verify my enrollment.
- US-4 View Transcript: As a student, I want to view my transcript so that I can review completed courses and GPA.

5.2 Sprint 2: Analytics and Finance

- US-A Attendance Tracking: As a student, I want to review my attendance per course and date so that I can avoid falling below required thresholds.
- US-M Marks Breakdown: As a student, I want to inspect my marks per category so that I understand my performance in each course.
- US-F Fee Challan: As a student, I want to view my fee challan so that I can understand my payable dues.

5.3 Sprint 3: Roles, Admin, Faculty, Feedback

- US-301 (any user): I want a landing page that lets me pick my role so that I can reach the correct login screen.
- US-302 (any user): I want to sign in with my role-specific credentials so that I only see the pages I am authorised to access.
- US-303 (platform owner): I want route guards to enforce role authorisation so that cross-role URL tampering is blocked.
- US-304 (admin): I want a dashboard with system-wide stat cards so that I can see student, faculty, and assignment counts at a glance.
- US-305 (admin): I want to assign instructors to course-sections so that faculty workflows have a source of assignments.
- US-306 (admin): I want to search and edit student profiles so that roll-number lookups stay accurate.
- US-307 (admin): I want to edit faculty profiles so that department and designation changes are reflected system-wide.
- US-308 (faculty): I want a dashboard showing my assigned sections and feedback dashboards so that I can navigate to my teaching tasks.
- US-309 (faculty): I want to upload attendance per session so that students see live attendance records in their dashboard.
- US-310 (faculty): I want to upload marks by assessment category so that students see accurate marks breakdowns and weighted contributions.

- US-311 (faculty): I want to see aggregated anonymous feedback for each of my sections so that I can improve my teaching without compromising student anonymity.
- US-312 (student): I want to submit anonymous 1 to 5 feedback on five standardised questions for each of my enrolled course-sections so that I can influence teaching quality without being identified.

Sprint Carryover

Sprint 1 carried View Transcript refinements into Sprint 2 where they were completed. The Sprint 2 deliverable closed with no carryover; all Sprint 2 user stories were marked complete. Advisor-oriented analytics initially noted in Sprint 2 planning were absorbed into the Sprint 3 Faculty Feedback Insights view.

6. Structured Specifications

6.1 Register Course

Actor: Student
Precondition: Student logged in
Postcondition: Course added to registration
Main Flow:

1. Student selects register option.
2. System displays available courses.
3. Student selects course.
4. System checks seat availability.
5. System saves registration.
6. Confirmation displayed.

Alternate Flow:

- If seats full: Error message displayed.
- If already registered: Warning displayed.

6.2 Drop Course

Actor: Student
Precondition: Student has registered courses
Main Flow:

1. Student selects drop option.
2. System displays registered courses.
3. Student selects course.
4. System removes course.
5. Confirmation displayed.

6.3 View Registered Courses

Actor: Student
Main Flow:

1. Student selects My Courses.
2. System retrieves enrolled courses.
3. System displays course details.

6.4 View Transcript

Actor: Student

Main Flow:

1. Student selects transcript option.
2. System retrieves completed courses and grades.
3. System calculates GPA.
4. System displays transcript.

6.5 Faculty Upload Attendance

Actor: Faculty

Precondition: Faculty assigned to the course-section

Main Flow:

1. Faculty selects an assigned course-section and a class date.
2. System renders the date by student matrix sorted by roll number.
3. Faculty marks each student Present, Late, or Absent.
4. System saves the attendance row transactionally.
5. Student dashboards reflect the new session.

6.6 Admin Assign Instructor

Actor: Admin

Main Flow:

1. Admin picks Course, Section, Term, and Faculty.
2. System validates that no duplicate course+section assignment exists.
3. System creates the assignment row.
4. Grid below the form lists every active course-instructor-section pairing.

6.7 Submit Anonymous Feedback

Actor: Student

Precondition: Student is enrolled in the course-section

Main Flow:

1. Student selects an enrolled course-section.
2. Student submits a 1 to 5 rating on each of the five standardised questions.
3. System stores the response without student identity and replaces any prior submission.
4. Faculty Feedback Insights view aggregates the response into per-question averages.

7. Non-Functional Requirements

The system shall meet the following refined non-functional requirements:

Performance

- Respond to user actions within 2 seconds under normal load.
- Render attendance and grade upload matrices for course-sections of 50 or more students without noticeable UI lag.
- Efficiently execute multi-course queries for dashboards, marks, attendance, and feedback aggregates.

Usability

- Provide a role-tailored drawer and dashboard so that students, faculty, and admins each see only the links they are authorised to use.
- Use clear Vaadin components (grids, stat cards, status badges) and colour-coded attendance cells (green = Present, yellow = Late, red = Absent).

- Display meaningful empty-state and error messages, for example "Please sign in again to view your challan" and "Select both course and faculty".
- Support a dark / light theme toggle that persists during the session.

Reliability

- Ensure accurate calculations for GPA, attendance percentages, weighted marks, and fee totals.
- Save attendance and marks matrices transactionally so that partial saves cannot corrupt a session.
- Gracefully handle missing or incomplete data, for example showing zero responses rather than crashing when no feedback exists yet.
- Maintain referential integrity across enrolment, assignment, attendance, marks, and feedback tables through PostgreSQL foreign keys.

Security

- Enforce role-based access control via the AllowedRoles annotation and MainLayout.beforeEnter route guard.
- Bind authenticated user identity to the Vaadin session via SessionService and clear it on sign-out.
- Validate all inputs at the service layer (rating must be 1 to 5; assignment section must match enrolment section).
- Never expose student identities in faculty feedback views; only aggregated counts and averages are returned.
- Reject invalid role parameters in login URLs with a clear error message.
- Prevent duplicate course registration.

Scalability and Maintainability

- Implemented in Java using object-oriented principles, modular and maintainable for future extensions.
- Support an increasing number of students, faculty, and course-sections through repository-driven persistence.
- Allow new modules or roles to be added with minimal refactoring thanks to a clean package layout (model / repository / service / controller / ui.views).
- Avoid hard-coded user lists; all roles are seeded through students, faculty, and admins tables that grow organically.

8. Database Schema and Evolution

The relational schema grew across the three sprints in line with the new modules. Sprint 1 established the student, course, enrolment, and grade tables. Sprint 2 added attendance, marks-category, and fee-challan tables. Sprint 3 introduced the role tables and feedback subsystem and adjusted the enrolments table to support per-section faculty matching.

Sprint 3 Schema Extensions

- New tables: faculty, admins, course_instructor_assignments, feedback_questions, faculty_feedback, faculty_feedback_responses.
- Modified table: enrolments now carries a section column so that faculty assignments can be matched to student enrolments.
- Seed data for 1 faculty, 1 admin, 3 demo students, 5 standard feedback questions, and 5 faculty-course assignments.

9. Scrum Methodology and Sprint Tracking

All three sprints were planned and tracked using Trello with the following lists: Product Backlog, Sprint Backlog, To Do, In Progress (Doing), Testing, Done, and Leftover. Three snapshots were taken per sprint: at sprint planning, mid-sprint, and at the end of the sprint, providing visible burn-down evidence.

Sprint 1 Board Snapshots

At planning, the To Do column held Register Course, View Transcript, Drop Course, View Registered Courses, and Database Setup. By mid-sprint Database Setup had reached Done, Register Course had moved to Doing, and View Transcript and Drop Course had been pulled into the active tracks. By end of sprint Database Setup, Register Course, View Registered Courses, and Drop Course were all in Done; View Transcript was carried to the Leftover list for refinement in Sprint 2.

Sprint 2 and Sprint 3 Boards

Sprint 2 closed with all stories (Attendance, Marks Breakdown, Fee Challan) in Done and no carryover. Sprint 3 closed with all twelve role / admin / faculty / feedback stories in Done. Sprint 3 also absorbed the advisor-oriented analytics that had been mentioned during Sprint 2 planning, delivering them as the Faculty Feedback Insights view.

10. Testing and Quality Assurance

10.1 Test Plan

Scope: All three sprint deliveries were validated through two complementary test campaigns: Black-box (specification and requirements based) and White-box (code-structure driven).

Environment:

- Language / Runtime: Java 17.
- Frameworks: Spring Boot 3.2.3, Vaadin 24.4.x, JPA / Hibernate.
- Database: PostgreSQL (Neon).
- Unit testing: JUnit 5 with Mockito.
- Web-layer testing: MockMvc (slice and standalone).
- Manual / UI testing: Chrome, Firefox, Edge, Safari (JavaScript enabled).
- Default deployment: http://localhost:8080/.

Approach:

1. Author the black-box specification first to capture expected UI and workflow behaviour per role.
2. Design white-box tests against the internal branches, loops, and guards in the service, controller, and UI-routing layers.
3. Execute the white-box suite iteratively; fix defects, rerun, repeat until 100% green.
4. Use the resulting green white-box suite as a baseline, then execute the black-box cases for acceptance validation.

In scope: Web UI flows, session behaviour, role-based access, student academic self-service, faculty teaching operations, admin setup, cross-role end-to-end flows.

Out of scope: Load / performance testing, database inspection during tests, visual UI pixel correctness, generated Vaadin framework artefacts.

10.2 Black-Box Test Cases

Seventy-six black-box test cases were designed across eight functional areas. The table below summarises the distribution.

Test Area	Prefix	Cases	Priority Mix

Test Area	Prefix	Cases	Priority Mix
Authentication & Entry	BB-ENT	8	6x P1, 2x P2
Access Control & Session	BB-SEC	10	9x P1, 1x P2
Student Dashboard	BB-STU	2	1x P1, 1x P2
Course Catalog	BB-STU	5	3x P1, 2x P2
My Enrollments	BB-STU	3	2x P1, 1x P2
Attendance / Marks / Transcript	BB-STU	3	3x P1
Fee Challan	BB-STU	2	1x P1, 1x P2
Faculty Feedback (student side)	BB-STU	3	3x P1
Faculty Role	BB-FAC	5	5x P1
Administrator Role	BB-ADM	5	4x P1, 1x P2
Cross-Role / End-to-End	BB-E2E	3	2x P1, 1x P2
Boundary Value Analysis	BB-BVA	12	9x P1, 3x P2
Error Guessing	BB-EG	15	1x P1, 13x P2, 1x P3
Total		76	43 P1 / 32 P2 / 1 P3

Representative Test Cases

ID	Title	Expected Result
BB-ENT-005	Valid student credentials login	Success notification and redirect to student dashboard
BB-ENT-006	Wrong password	Invalid credentials error shown; no session created
BB-SEC-002	Student attempts /admin/dashboard	Redirected to student home; admin dashboard not rendered
BB-STU-014	Student registers same course twice	Student already enrolled in course error
BB-STU-016	Register when active credits + new course > 18 CH	Credit limit exceeded (max 18 CH) error
BB-STU-071	Fee challan page opened after session expires	Please sign in again to view your challan message
BB-FAC-003	Faculty adds an assessment column and marks	New column persisted; student rows become editable; notification on save
BB-FAC-004	Faculty uploads attendance via P / L / A matrix	Matrix saves successfully; student dashboard reflects new sessions
BB-ADM-002	Admin creates course-section-faculty assignment	Save succeeds; grid shows new row with course, section, instructor, term
BB-ADM-005	Duplicate assignment (same course+section)	Error message shown per app rules; no duplicate row inserted
BB-E2E-001	Admin assigns to Faculty uploads to Student views	All three roles see consistent, live data across their respective views

10.3 White-Box Testing Evidence

A 65-test white-box suite was authored against the internal code structure with the following coverage criteria.

Criterion	Representative Test IDs	Coverage Notes
Statement	EN-01, TR-04, AP-01, UI-07	Core service and controller methods execute without dead code in tested scope

Criterion	Representative Test IDs	Coverage Notes
Branch	AR-02, EN-03, UI-01, AU-02	Both decision outcomes exercised for major guards
Condition	AR-03, SF-02, UI-03, SF-03	Multi-condition guards validated with edge input combinations
Loop	AR-01, TR-01, CO-01, SF-01	Semester (1..8), list-filter, and aggregation loops covered
Selected Path	AR-04, TR-02, EN-05	Multi-step business-rule chains validated end-to-end

Module Coverage

- AcademicRulesService (AR-01..AR-06): semester progress loop, registration guards, credit and prerequisite conditions, promotion gate path, repeat-flag refresh.
- EnrollmentService (EN-01..EN-05): list, register-existing-enrolled, register-existing-dropped (re-activation), capacity check, drop rules.
- TranscriptService (TR-01..TR-04): semester loop, GPA/credits fallback, null grade handling, PDF export path.
- CourseService (CO-01..CO-02): catalog grouping by semester, available-course shortlist with backlog merge.
- FeeChallanService (FC-01..FC-04): DB-backed path, computed path, lab surcharge and fixed adjustments, missing-student branch.
- StudentFeedbackService (SF-01..SF-04): eligible-assignment filter, section-ownership validation, rating validation, missing reference branches.
- AuthService (AU-01..AU-03): role parsing, credential validation, per-role success branches.
- UI / Session / Route Guards (UI-01..UI-07): login parameter parsing, authenticated-reroute, beforeEnter guard, theme toggle, role resolver mapping, AllowedRoles annotation, session store / clear / retrieve.
- Controllers and Global Exception Handler (AP-01..AP-03): endpoint delegation, response status branches, exception-to-HTTP mapping.

10.4 Bug Report Summary

Six defects were discovered during the white-box campaign. All were fixed and re-verified in the final consolidated run.

ID	Area	Description	Resolution	Status
WB-01	SessionService	Session store failed without an active Vaadin session	Added session requirement check and failure message	Fixed
WB-02	AcademicRulesService tests	saveAll mismatch / cast failures in tests	Generic iterable-safe stubbing and fixture cleanup	Fixed
WB-03	CourseService test	Unnecessary stubbing failures in strict mode	Removed non-required stubs	Fixed
WB-04	LoginView test	Unresolved UI type in event helper	Corrected UI reference and helper fixture	Fixed
WB-05	MainLayout test	Implicit router unavailable in unit path	Guarded route assignment and stabilised fixture behaviour	Fixed

ID	Area	Description	Resolution	Status
WB-06	Controller tests	Harness instability with environment / tooling constraints	Deterministic controller harness and explicit path variables	Fixed

Stabilisation History

Stage	Result	Notes
Initial post-annotation rerun	20 passed / 13 failed	Fixture and strict-stubbing regressions detected
Mid-repair run	28 passed / 5 failed	Core service failures resolved; UI / router issues isolated
UI stabilisation run	6 passed / 0 failed	LoginView and MainLayout fixture stabilisation complete
Controller stabilisation run	4 passed / 0 failed	Controller-specific test harness stabilised
Final consolidated run	65 passed / 0 failed	White-box scope fully green

10.5 Test Results and Observations

- White-box suite: 65 / 65 passed (100% pass rate) after 5 stabilisation stages.
- Black-box specification: 76 cases authored and executed across all eight functional areas; all critical-path (P1) cases passed.

Key Observations

- Session safety was improved dramatically: the WB-01 fix means that any accidental background call that depends on a missing Vaadin session now produces a clean error instead of a silent failure.
- Rule-engine rigor was verified: branch coverage on AcademicRulesService (prerequisites, credit cap, promotion, drop eligibility) caught corner cases that would otherwise have surfaced only during end-of-semester transitions.
- Controller exception mapping is consistent: every thrown service exception maps to a deterministic HTTP response, keeping the UI's error messages coherent across all views.
- Role guards are airtight: UI-03 and BB-SEC-002..004 both confirm that no role can reach another role's pages through URL tampering.

How Testing Improved System Quality

1. Silent failures eliminated: the SessionService guard (WB-01) replaced a silent store-miss with an explicit, user-visible error.
2. Test harness hygiene: removing over-specific mocks (WB-02, WB-03) cleaned up dead stubs and revealed two academic-rule edge cases that had been hidden behind lenient fixtures.
3. UI reliability: LoginView (WB-04) and MainLayout (WB-05) fixture fixes stabilised the routing behaviour exercised on every navigation, reducing intermittent failures to zero.
4. Deterministic controller responses: the WB-06 harness fix means controller tests now pin HTTP status codes, preventing future regressions from altering response codes unnoticed.
5. Cross-role safety validated: black-box access-control cases and the white-box beforeEnter tests together mean that no single role can see another role's data, which is the cornerstone of a multi-role business system.

11. Work Division Across Sprints

11.1 Sprint 1

Team Member	Responsibility
Syed Muhammad Mustafa	Database design and integration
Talha and Muhammad	Course registration and drop functionality
Mustafa and Talha	Transcript module and testing
Talha Qamar	Sprint planning, Trello updates, burndown tracking

11.2 Sprint 2

Team Member	Responsibility
Syed Mustafa	Attendance Tracking module (backend + UI)
Talha Qamar	Marks Breakdown module (logic + calculations)
Muhammad Bin Taimur	Fee Challan module (DB + UI + integration)
Syed Mustafa	Trello management, sprint tracking, testing coordination

All members collaborated on database updates, testing, and UI improvements.

11.3 Sprint 3

Team Member	Primary Responsibility
Syed Muhammad Mustafa (23I-0661)	Faculty module end-to-end: FacultyDashboardView, FacultyAttendanceUploadView, FacultyGradeUploadView, FacultyFeedbackInsightsView, and FacultyWorkflowService (roster, attendance upload, assessment save, feedback aggregation).
Muhammad Talha Qamar (23I-0013)	Admin module end-to-end: AdminDashboardView, AdminAssignmentView, AdminManageUsersView, and AdminManagementService. White-box testing (65-test JUnit / Mockito suite, coverage-criteria design, defect triage and fixes WB-01 through WB-06). Trello / sprint tracking and burn-down.
Muhammad Bin Taimur (23I-0810)	Student Feedback module: StudentFeedbackView, StudentFeedbackService. Role-based Authentication: LoginView, RoleSelectionView, AuthService, SessionService, AllowedRoles annotation, RoleRouteResolver. Black-box testing (76-case specification across BB-ENT / BB-SEC / BB-STU / BB-FAC / BB-ADM / BB-E2E / BB-BVA / BB-EG).

Shared responsibilities: Database schema extensions (faculty, admins, course_instructor_assignments, feedback_questions, faculty_feedback, faculty_feedback_responses), seed data for the new tables, cross-role integration smoke testing, UI theming polish, and deliverable documentation.

12. Final System Summary

The Student Portal System evolved over three sprints from a single-role course-management utility into a complete three-role academic business management platform.

- Sprint 1 laid the foundation by delivering the student-facing course lifecycle (Course Registration, Course Drop, View Registered Courses, View Transcript with GPA). This established the domain model and the core service layer that every subsequent sprint built upon.
- Sprint 2 extended the platform with academic self-service analytics and financial transparency

(Attendance Tracking, Marks Breakdown, Fee Challan). Students could now monitor trends, understand category-wise performance, and see itemised financial obligations without any external paperwork.

- Sprint 3 completed the system by adding the producers and administrators that the prior sprints had implied but not implemented. The new Faculty role uploads the attendance and marks data that Sprint 2's views consume; the new Admin role provisions the course-to-faculty assignments that make those uploads possible; and the new Student Feedback module closes the loop with anonymous, aggregated instructor feedback. Role-based authentication binds the three roles safely together under a single Vaadin application.

Final Status

Every user story planned across the three sprints has been delivered. The system is covered by a white-box test suite that passed 65 / 65 (100%) and a black-box specification of 76 test cases across eight functional areas, with all critical-path (P1) cases executed. Six defects were discovered, fixed, and re-verified during Sprint 3. The system is feature-complete against the original product vision: a centralised, efficient, user-friendly academic management platform for students, faculty, and administrators.